

LAPORAN PRAKTIKUM
MODUL 10
Pemrograman Berorientasi Objek

Disusun oleh:

Paulus Roberto Apolos Siagian

NIM: 105224020

BAB I PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan paradigma pemrograman yang mengorganisasi desain perangkat lunak di sekitar data atau objek, bukan fungsi dan logika. Dalam paradigma ini, setiap objek dapat menerima pesan, memproses data, dan mengirimkan pesan ke objek lain.

Pada praktikum Modul 10 ini, konsep PBO diterapkan dalam pembuatan Sistem Gudang menggunakan bahasa pemrograman Java. Program ini dirancang untuk mengelola data barang di sebuah gudang, termasuk penambahan barang baru, penambahan stok, pengurangan stok, serta pencetakan laporan akhir.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum Modul 10 ini adalah:

1. Memahami dan mengimplementasikan konsep kelas dan objek dalam Java.
2. Menerapkan penggunaan Map, Set, dan List sebagai struktur data dalam program Java.
3. Membangun sistem manajemen gudang sederhana berbasis OOP (Object-Oriented Programming).
4. Melatih kemampuan pembuatan metode (method) dalam sebuah kelas Java.

BAB II DASAR TEORI

2.1 Pemrograman Berorientasi Objek (OOP)

Object-Oriented Programming (OOP) adalah paradigma pemrograman yang memandang program sebagai kumpulan objek yang saling berinteraksi. Setiap objek memiliki atribut (properti) dan perilaku (metode). Empat pilar utama OOP adalah Encapsulation, Inheritance, Polymorphism, dan Abstraction.

2.2 Kelas dan Objek dalam Java

Kelas (class) adalah blueprint atau cetak biru dari sebuah objek. Kelas mendefinisikan atribut dan metode yang dimiliki oleh objek. Objek adalah instansi dari sebuah kelas yang dibuat menggunakan kata kunci new. Pada praktikum ini, terdapat dua kelas utama yaitu kelas Barang dan kelas SistemGudang.

2.3 Collection Framework (Map, Set, List)

Java Collection Framework menyediakan berbagai struktur data yang siap pakai. HashMap digunakan untuk menyimpan pasangan key-value dan memungkinkan akses data secara efisien berdasarkan kunci. HashSet digunakan untuk menyimpan kumpulan nilai unik tanpa duplikasi. ArrayList digunakan untuk menyimpan daftar data yang terurut dan dapat diakses berdasarkan indeks.

2.4 Exception Handling

Exception handling adalah mekanisme dalam Java untuk menangani kesalahan atau kondisi tidak normal yang terjadi saat program berjalan. Menggunakan blok try-catch, program dapat mendeteksi dan menangani pengecualian tanpa menyebabkan program berhenti secara tiba-tiba. Pada program ini, exception handling diterapkan pada metode pengurangan stok untuk menghindari stok bernilai negatif.

BAB III PEMBAHASAN DAN HASIL

3.1 Kelas Barang

Kelas Barang merupakan representasi dari sebuah barang yang ada di gudang. Kelas ini mendefinisikan atribut dan perilaku dasar yang dimiliki oleh setiap barang. Berikut adalah kode program untuk kelas Barang:

```
class Barang {  
    private String idBarang;  
    private String namaBarang;  
    private String kategori;  
    private int stok;  
  
    public Barang(String idBarang, String namaBarang, String kategori, int stok) {  
        this.idBarang = idBarang;  
        this.namaBarang = namaBarang;  
        this.kategori = kategori;  
        this.stok = stok;  
    }  
  
    public void tambahStokEkstra(int jumlah) {  
        this.stok += jumlah;  
    }  
  
    public void kurangiStokEkstra(int jumlah) {  
        this.stok -= jumlah;  
    }  
  
    public void infoBarang() {  
        System.out.println("ID: " + this.idBarang + " | Nama: " + this.namaBarang + " | Kategori: " + this.kategori + " | Stok: " + this.stok);  
    }  
}
```

Gambar 3.1 Kode Program Kelas Barang

Penjelasan kode pada kelas Barang:

Kelas Barang memiliki empat atribut private yaitu idBarang (String), namaBarang (String), kategori (String), dan stok (int). Penggunaan modifier private menerapkan konsep encapsulation sehingga atribut hanya dapat diakses melalui metode yang disediakan.

Constructor Barang menerima empat parameter: idBarang, namaBarang, kategori, dan stok. Di dalam constructor, setiap parameter diassign ke atribut kelas menggunakan kata kunci this untuk membedakan antara variabel lokal dan atribut kelas.

Metode tambahStokEkstra(int jumlah) digunakan untuk menambah jumlah stok barang. Operasi this.stok += jumlah berarti nilai stok saat ini ditambah dengan nilai jumlah yang diberikan sebagai parameter.

Metode kurangiStokEkstra(int jumlah) digunakan untuk mengurangi jumlah stok barang. Operasi this.stok -= jumlah berarti nilai stok saat ini dikurangi dengan nilai jumlah yang diberikan.

Metode `infoBarang()` digunakan untuk menampilkan informasi lengkap dari sebuah barang, meliputi ID, nama, dan kategori barang ke layar konsol menggunakan `System.out.println`.

3.2 Kelas SistemGudang

Kelas `SistemGudang` adalah kelas utama yang mengelola keseluruhan operasi gudang. Kelas ini menggunakan berbagai struktur data dari Java Collection Framework untuk mengelola barang. Berikut adalah kode program kelas `SistemGudang`:

```

import java.util.Map;
import java.util.HashMap;
import java.util.Set;
import java.util.HashSet;
import java.util.List;
import java.util.ArrayList;

class SistemGudang {
    private Map<String, Barang> databaseBarang;
    private Set<String> kategoriUnik;
    private List<String> riwayat;

    public SistemGudang() {
        this.databaseBarang = new HashMap<>();
        this.kategoriUnik = new HashSet<>();
        this.riwayat = new ArrayList<>();
    }

    public void tambahBarangBaru(String id, String nama, String kategori, int stok) {
        if (!databaseBarang.containsKey(id)) {
            Barang barangBaru = new Barang(id, nama, kategori, stok);
            databaseBarang.put(id, barangBaru);
            kategoriUnik.add(kategori);
            riwayat.add("Barang Baru Terdaftar: " + id + " (" + nama + ") sejumlah " + stok + " unit.");
        } else {
            riwayat.add("Gagal Daftar: ID " + id + " sudah ada di database.");
        }
    }

    public void tambahStok(String id, int jumlah) {
        Barang barang = databaseBarang.get(id);
        if (barang != null) {
            barang.tambahStokEkstra(jumlah);
            riwayat.add("Barang Masuk: " + id + " ditambah " + jumlah + " unit.");
        } else {
            riwayat.add("Gagal Tambah Stok: ID " + id + " tidak ditemukan.");
        }
    }

    public void kurangiStok(String id, int jumlah) {
        Barang barang = databaseBarang.get(id);
        if (barang != null) {
            try {
                barang.kurangiStokEkstra(jumlah);
                riwayat.add("Barang Keluar: " + id + " dikurangi " + jumlah + " unit.");
            } catch (Exception e) {
                riwayat.add("Gagal Kurangi Stok: ID " + id + " stok tidak mencukupi.");
            }
        } else {
            riwayat.add("Gagal Kurangi Stok: ID " + id + " tidak ditemukan.");
        }
    }

    public void kurangiStokAman(String id, int jumlah, int stokSaatIni) {
        Barang barang = databaseBarang.get(id);
        if (barang != null) {
        }
    }

    public void kurangiStokSimulasi(String id, int jumlah, boolean sukses) {
        Barang barang = databaseBarang.get(id);
        if (barang != null && sukses) {
            barang.kurangiStokEkstra(jumlah);
            riwayat.add("Barang Keluar: " + id + " dikurangi " + jumlah + " unit.");
        } else {
            riwayat.add("Barang Keluar GAGAL: " + id + " ditarik " + jumlah + " unit (Stok tidak mencukupi atau ID salah).");
        }
    }

    public void cetakLaporan() {
        System.out.println("=== LAPORAN AKHIR SISTEM GUDANG ===");

        System.out.println("\n[DAFTAR KATEGORI UNIK]");
        for (String kat : kategoriUnik) {
            System.out.println("- " + kat);
        }

        System.out.println("\n[SISA STOK BARANG]");
        for (Barang b : databaseBarang.values()) {
            b.infoBarang();
        }

        System.out.println("\n[URUTAN RIWAYAT TRANSAKSI]");
        int no = 1;
        for (String log : riwayat) {
            System.out.println(no + ". " + log);
            no++;
        }
        System.out.println("=====");
    }
}

```

Gambar 3.2 Kode Program Kelas SistemGudang

Penjelasan kode pada kelas SistemGudang:

Kelas SistemGudang memiliki tiga atribut utama: databaseBarang bertipe Map<String, Barang> yang menggunakan HashMap untuk menyimpan data barang dengan ID sebagai kunci; kategoriUnik bertipe Set<String> yang menggunakan HashSet untuk menyimpan kategori barang secara unik tanpa duplikasi; dan riwayat bertipe List<String> yang menggunakan ArrayList untuk menyimpan riwayat seluruh transaksi yang terjadi.

Metode tambahBarangBaru() memeriksa apakah ID barang sudah ada di database menggunakan containsKey(). Jika belum ada, barang baru dibuat dan disimpan ke databaseBarang, kategorinya ditambahkan ke kategoriUnik, dan riwayat transaksi dicatat. Jika ID sudah ada, pesan gagal dicatat ke riwayat.

Metode tambahStok() mengambil objek Barang dari database berdasarkan ID, kemudian memanggil metode tambahStokEkstra() dari kelas Barang untuk menambah stok. Setiap operasi berhasil atau gagal dicatat ke dalam riwayat.

Metode kurangiStok() menggunakan blok try-catch untuk menangani kemungkinan kegagalan pengurangan stok. Jika pengurangan berhasil, riwayat dicatat sebagai sukses. Jika terjadi exception (stok tidak mencukupi), pesan gagal dicatat ke riwayat.

Metode kurangiStokSimulasi() mensimulasikan pengurangan stok berdasarkan parameter sukses (boolean). Jika sukses bernilai true dan barang ditemukan, stok dikurangi dan riwayat berhasil dicatat. Jika tidak, pesan gagal dicatat.

Metode cetakLaporan() menampilkan laporan akhir sistem gudang yang meliputi daftar kategori unik, sisa stok setiap barang (menggunakan infoBarang()), dan urutan seluruh riwayat transaksi yang telah dilakukan.

3.3 Kelas Main (Program Utama)

Kelas Main merupakan titik masuk (entry point) dari program yang berisi metode main(). Di sinilah objek SistemGudang dibuat dan seluruh operasi dijalankan secara berurutan. Berikut adalah kode program kelas Main:

```
public class Main {  
    Run | Debug  
    public static void main(String[] args) {  
        SistemGudang gudang = new SistemGudang();  
  
        gudang.tambahBarangBaru(id: "B01", nama: "Laptop ASUS", kategori: "Elektronik", stok: 10);  
        gudang.tambahBarangBaru(id: "B02", nama: "Meja Kerja", kategori: "Furnitur", stok: 5);  
        gudang.tambahBarangBaru(id: "B03", nama: "Mouse Logitech", kategori: "Elektronik", stok: 15);  
  
        gudang.tambahStok(id: "B01", jumlah: 5);  
        gudang.kurangiStokSimulasi(id: "B02", jumlah: 3, sukses: true);  
        gudang.kurangiStokSimulasi(id: "B02", jumlah: 10, sukses: false);  
        gudang.cetakLaporan();  
    }  
}
```

Gambar 3.3 Kode Program Kelas Main

Penjelasan kode pada kelas Main:

Sebuah objek SistemGudang bernama gudang dibuat menggunakan new SistemGudang(). Objek ini kemudian digunakan untuk memanggil seluruh operasi yang tersedia.

Tiga barang ditambahkan ke dalam sistem: Laptop ASUS (ID: B01, kategori Elektronik, stok 10 unit), Meja Kerja (ID: B02, kategori Furnitur, stok 5 unit), dan Mouse Logitech (ID: B03, kategori Elektronik, stok 15 unit). Karena B01 dan B03 sama-sama berkategori "Elektronik", HashSet pada kategoriUnik hanya akan menyimpan satu entri "Elektronik".

Stok Laptop ASUS (B01) ditambah sebanyak 5 unit menjadi 15 unit menggunakan tambahStok(). Kemudian dilakukan dua simulasi pengurangan stok pada Meja Kerja (B02): pertama dengan jumlah 3 unit dan parameter sukses true (berhasil, stok menjadi 2), kedua dengan jumlah 10 unit dan parameter sukses false (sengaja digagalkan).

Terakhir, cetakLaporan() dipanggil untuk menampilkan seluruh hasil operasi, termasuk kategori unik, sisa stok semua barang, dan urutan riwayat transaksi.

BAB IV KESIMPULAN

Berdasarkan praktikum Modul 10 yang telah dilaksanakan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Program Sistem Gudang berhasil dibangun menggunakan konsep Pemrograman Berorientasi Objek (OOP) dengan bahasa Java, yang terdiri dari tiga kelas utama: Barang, SistemGudang, dan Main.
2. Kelas Barang berhasil mengimplementasikan enkapsulasi data dengan menggunakan modifier private pada atributnya, serta menyediakan metode untuk memanipulasi data stok barang.
3. Kelas SistemGudang berhasil memanfaatkan Collection Framework Java secara tepat: HashMap untuk penyimpanan data barang, HashSet untuk menjaga keunikan kategori, dan ArrayList untuk pencatatan riwayat transaksi.
4. Penerapan exception handling dengan blok try-catch pada metode pengurangan stok memastikan program dapat menangani kondisi stok tidak mencukupi tanpa crash.
5. Kelas Main sebagai program utama berhasil mendemonstrasikan seluruh fitur sistem gudang, mulai dari penambahan barang, manipulasi stok, hingga pencetakan laporan akhir.